

Universidad de Las Palmas de Gran Canaria

Grado en Ciencia e Ingeniería de Datos

Big Data: Search Engine Project

Stage 1: Building the Data Layer

Course: Big Data
Group: PIE data
Repository: https://github.com/PIE-data/stage_1

Group members

Marcela Nieto-Kuczyńska
Andrea Patruno
Ivan Colella
Manuel Muñoz Roca

Academic Year 2026/2027

Contents

2	Introduction and Objectives	1
3	System Architecture	1
3.1	Datalake	1
3.2	Datamarts	2
3.3	Control Layer	2
4	Design Decisions	2
4.1	Programming Languages: Python, Node.js, and Go	3
4.2	Index Backends: JSON, Folder, and SQLite	3
4.3	English-only Corpus	3
4.4	No Stemming	3
4.5	Word-level Positional Index	3
4.6	Local Mirror for Benchmarks	3
4.7	Metadata Store	3
5	Benchmarks and Results	4
5.1	Dataset, Methods, and Measures	4
5.2	Datalake Results	4
5.3	Inverted-index Results	8
5.4	Supplementary Metadata Datamart Results	10
5.5	Results Compared with Expectations	11
5.6	Threats to Validity	11
6	Conclusions and Future Improvements	11

2 Introduction and Objectives

The overall goal of the Big Data course project is to design and implement a simple search engine from scratch. Stage 1 focuses on the data layer: the part of the system responsible for acquiring documents, transforming them into consistent representations, storing them efficiently, and tracking the state of the pipeline.

The dataset is based on Project Gutenberg. Each book is downloaded as plain text and split using the Project Gutenberg start and end markers into a header and a cleaned body. The header is used to extract structured metadata, while the body is tokenized to build a word-level inverted index. The footer is not required by the later search pipeline and is discarded.

The objective of this stage is not only to build a working implementation, but also to compare alternative physical storage designs under controlled conditions. The pipeline is implemented in Python, Node.js, and Go, using the same corpora, normalization rules, workloads, and output contract in every language. This allows performance differences to be attributed to implementation and storage choices rather than to differences in functionality.

The result of Stage 1 is a reliable and reproducible storage architecture that separates unstructured book content from structured, queryable data. A control layer makes the pipeline restartable and idempotent. This foundation will be reused in Stage 2 for ranking, phrase queries, and more advanced search functionality.

3 System Architecture

The system architecture is structured into three distinct physical and logical layers: the Datalake (raw artifact storage), the Datamarts (structured metadata and indices), and the Control Layer (state tracking and concurrency). Figure 1 illustrates the data flow from ingestion to querying.

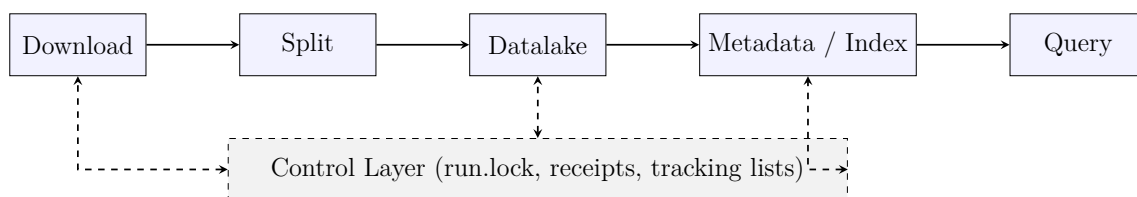


Figure 1: System pipeline and Control Layer interaction.

3.1 Datalake

The Datalake stores the split artifacts resulting from the ingestion of Gutenberg books. The system guarantees atomic writes and supports three mutually exclusive directory layouts:

- **Time-based layout:** artifacts are partitioned by UTC ingestion time: `datalake/<YYYYMMDD>/<HH>` and the corresponding header file.

- **Book-based layout:** artifacts are grouped by book ID: `datalake/books/<id>/body.txt`, `header.txt`, and `meta.json`.
- **Batch/prefix layout (CLI name hash):** artifacts are sharded using the zero-padded six-digit decimal book ID, with `id6[0:2]/id6[2:4]`. Despite the historical CLI name, this is a digit-prefix partitioning scheme, not a hash function.

3.2 Datamarts

The Datamarts transform the raw datalake files into structured queryable formats, separating metadata from the inverted index.

Metadata Store. The parsed header fields are stored in a centralized SQLite database at `datamarts/metadata.db`. The database uses Write-Ahead Logging (WAL) and stores book identity, parsed title/author/language fields, workspace-relative paths, byte counts, content hashes, and ingestion timestamps.

Inverted Index Backends. The positional inverted index stores document IDs, term frequencies, and token positions. Three interchangeable physical backends are implemented:

- **JSON:** one monolithic file at `datamarts/inverted_index.json`.
- **Folder:** one file per term under `datamarts/inverted_index/<BUCKET>/<safe_term>.txt`.
- **SQLite:** an embedded B-tree database at `datamarts/index.db`, using `terms` and `postings` tables.

3.3 Control Layer

The Control Layer governs execution state, restartability, and concurrency. All control artifacts live under `control/`:

- **State tracking lists:** `downloaded_books.txt` and `indexed_books.txt` record successfully completed stages, while `failed_books.txt` records failures and their reasons.
- **Concurrency lock:** `run.lock` prevents overlapping writers.
- **Ingestion receipts:** JSON records under `control/ingestion/<layout>/<id>.json` bind a book to its ingestion time and final artifact paths.
- **Reconciliation:** after a crash, the control state can be rebuilt from complete datalake artifacts, allowing the pipeline to converge to a consistent state.

4 Design Decisions

The implementation choices were made to maximize comparability, reproducibility, and the amount of useful benchmarking that could be completed within the project schedule.

4.1 Programming Languages: Python, Node.js, and Go

Python serves as the reference implementation. Node.js and Go were selected as the two additional languages because they provide meaningfully different runtime models while remaining lightweight to build and execute. Python provides a high-level baseline, Node.js uses V8 and an event-driven model, and Go provides a compiled implementation with lightweight concurrency. A shared CLI and specification keep the comparison focused on implementation behavior rather than semantics.

4.2 Index Backends: JSON, Folder, and SQLite

The three index structures represent three distinct storage strategies: one large file, many small files, and an embedded B-tree database. SQLite was chosen instead of MongoDB because the project permits custom approaches, SQLite requires no external service, and it is already used by the metadata datamart. This avoids introducing network and service-management overhead into the mandatory comparison.

4.3 English-only Corpus

The benchmark corpus is restricted to English-language books so that all implementations can use one fixed stop-word list and identical normalization rules. Controlled benchmark tiers of 100 and 1,000 books were completed. A planned 10,000-book scalability tier was not completed and is treated explicitly as a limitation in Section 5.6.

4.4 No Stemming

Stemming is excluded from Stage 1. Equivalent stemming libraries in Python, Node.js, and Go are not guaranteed to produce byte-identical output for every token. Because the project uses canonical export hashes as a strict conformance test, stemming would add an unnecessary source of cross-language divergence.

4.5 Word-level Positional Index

The index stores term frequency and token positions in addition to document IDs. This uses more storage than a record-level index, but it preserves information required for phrase queries and later ranking methods such as TF-IDF or BM25.

4.6 Local Mirror for Benchmarks

All benchmark downloads use a local HTTP mirror instead of the live Project Gutenberg service. This removes internet latency and remote rate limiting from the primary measurements so that the benchmark reflects the pipeline and storage design.

4.7 Metadata Store

Alternative metadata databases were optional, so the project keeps SQLite as the metadata store rather than comparing multiple metadata technologies. The implemented metadata pipeline was nevertheless benchmarked for build and query behavior as a supplementary check of the datamart.

5 Benchmarks and Results

5.1 Dataset, Methods, and Measures

Table 1: Benchmark design used for the Stage 1 evaluation.

Dataset	Method	Measures
20-book golden corpus	Python, Node.js, and Go; common specification and canonical export	Cross-language and cross-backend conformance
1,000-book English corpus	Language $\times$ datalake layout: time, book, batch/prefix	Download/write throughput; lookup latency; incremental-processing cost; recovery behavior; storage overhead
1,000-book English corpus	Language $\times$ index backend: JSON and SQLite	Index-build time; update time; peak RSS; time-memory trade-off
100-book English corpus	Language $\times$ folder index backend	Index-build and update cost; used to establish the practical limit of the one-file-per-term design
100 / 1,000 books	SQLite metadata build and metadata-query benchmark	Build latency; query latency; query plan behavior

The primary benchmark machine was an MSI laptop with an Intel i7-1255U, 16 GB RAM (10 GiB assigned to WSL2), NVMe SSD, and Ubuntu on WSL2 with an ext4 benchmark workspace. Python 3.14, Node 22, and Go 1.26 were used. All downloads came from the local mirror.

Whole CLI commands are measured by the external macro runner so that process startup, I/O, and child-process memory are captured consistently across languages. Operations too short for meaningful process-level timing—datalake lookup and query—use an internal micro-benchmark layer. Results are reported using medians and IQR rather than means and standard deviations because the distributions are right-skewed.

5.2 Datalake Results

Figure 2 shows download and write throughput for 1,000 books. Python and Go benefit strongly from increasing the worker count from one to eight. Go improves from approximately 42 s to 9.6–11.1 s depending on the layout, while Python improves from approximately 43–49 s to 14–16 s. Node remains around 67–80 s and receives little or no benefit from eight workers. This contradicted the natural expectation that additional workers would improve all three implementations; inspection of the implementation indicates that synchronous filesystem writes serialize work on Node’s event loop.

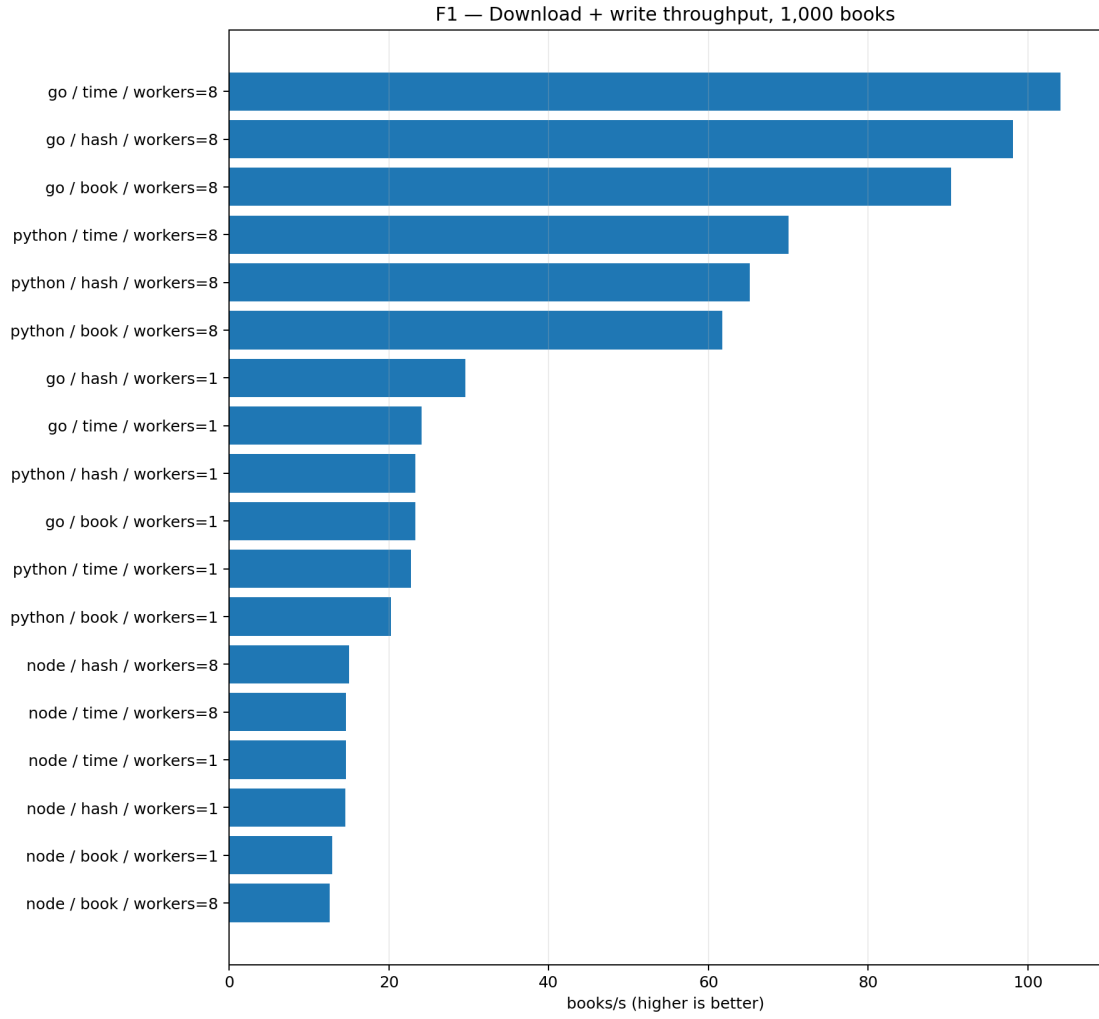


Figure 2: F1: download and write throughput for 1,000 books.

The Python micro-benchmark in Figure 3 shows that direct lookup is cheapest for the batch/prefix layout (median $75.6 \mu\text{s}$), followed by the book layout ($87.4 \mu\text{s}$). The time layout is substantially slower at $436.5 \mu\text{s}$ because lookup requires a directory walk. Although E2 micro measurements are available only for Python, the result exposes an architectural property of the layouts rather than merely a language effect.

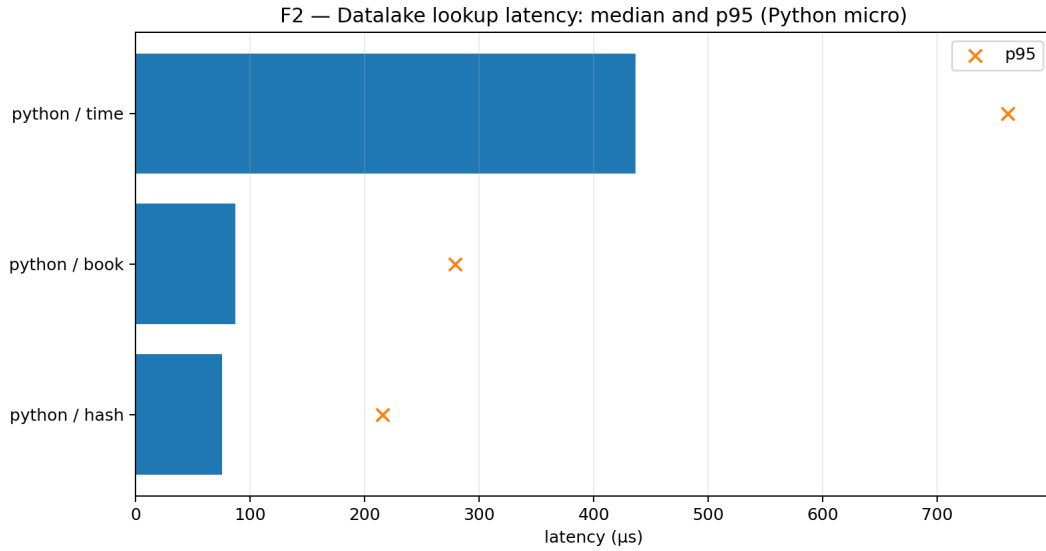


Figure 3: F2: datalake lookup latency (Python micro-benchmark).

Figure 4 compares incremental detection. The time-based layout is particularly efficient when the ingestion timestamp can narrow the scan, while the prefix and book layouts trade some scan cost for predictable direct lookup. Figure 5 evaluates recovery after a forced termination. All tested language/layout combinations recovered without losing or duplicating books, which is more important for this experiment than the small performance differences between layouts.

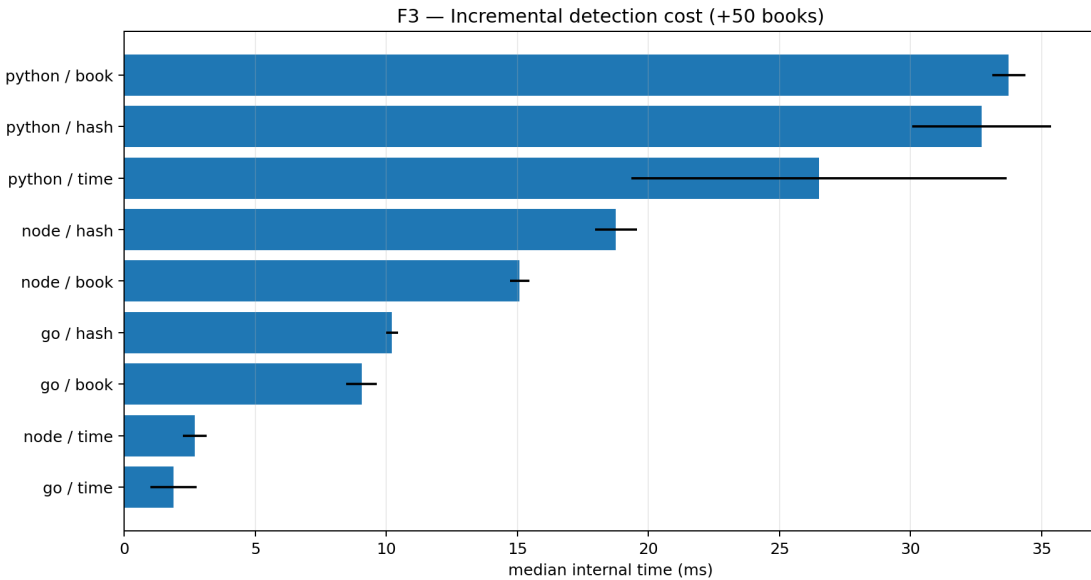


Figure 4: F3: incremental detection cost after adding 50 books.

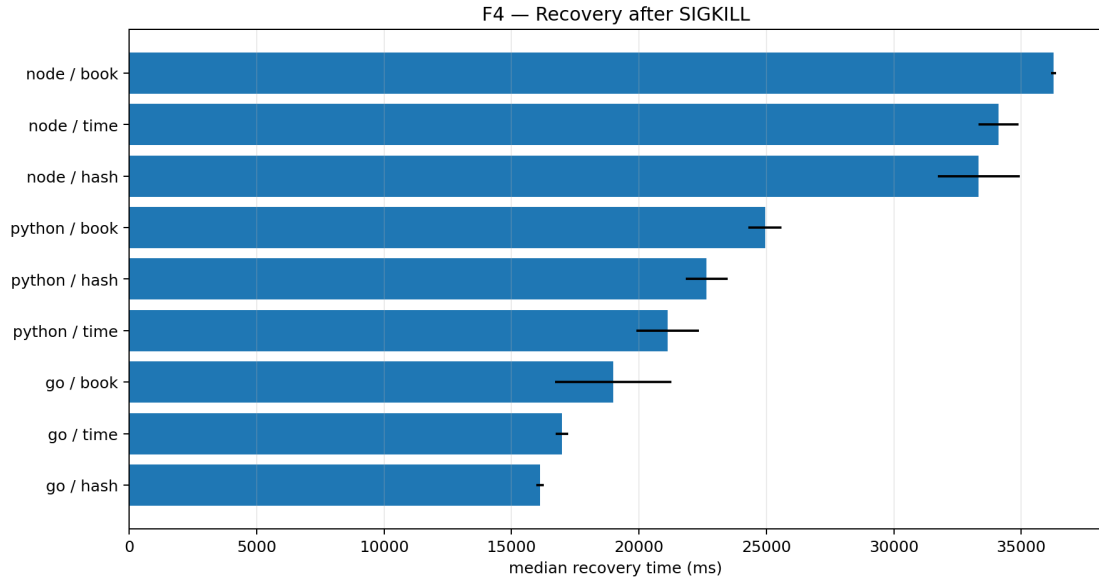


Figure 5: F4: recovery time after SIGKILL at approximately 50% progress.

Storage allocation is very similar for all three layouts. Figure 6 shows approximately 391 MiB for the time layout, 393 MiB for batch/prefix, and 399 MiB for book-based storage; the spread is below 2%. Storage footprint therefore does not decide the layout choice.

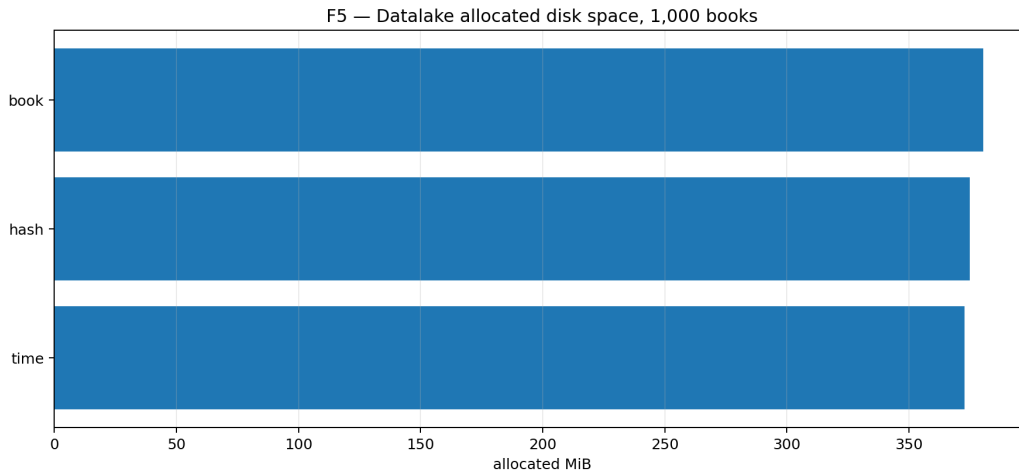


Figure 6: F5: allocated datalake storage for 1,000 books.

Selected datalake layout. We select the **batch/prefix layout** (named **hash** in the CLI) as the default. The time layout is attractive for incremental scans and slightly faster in some throughput runs, but it makes direct lookup much more expensive because a book ID does not identify its path. The book layout offers good lookup but creates one directory per book and has the largest storage allocation. Batch/prefix gives the best measured lookup, competitive parallel ingestion, bounded directory fan-out, and a deterministic path derived directly from the book ID. It is therefore the strongest overall compromise.

5.3 Inverted-index Results

Figure 7 compares index construction over 1,000 books for JSON and SQLite. Go with JSON is the fastest configuration at 29.4 s. Node takes 78.8 s with JSON and 79.4 s with SQLite. Python takes 109.5 s with JSON and 118.9 s with SQLite, while Go with SQLite takes 97.5 s.

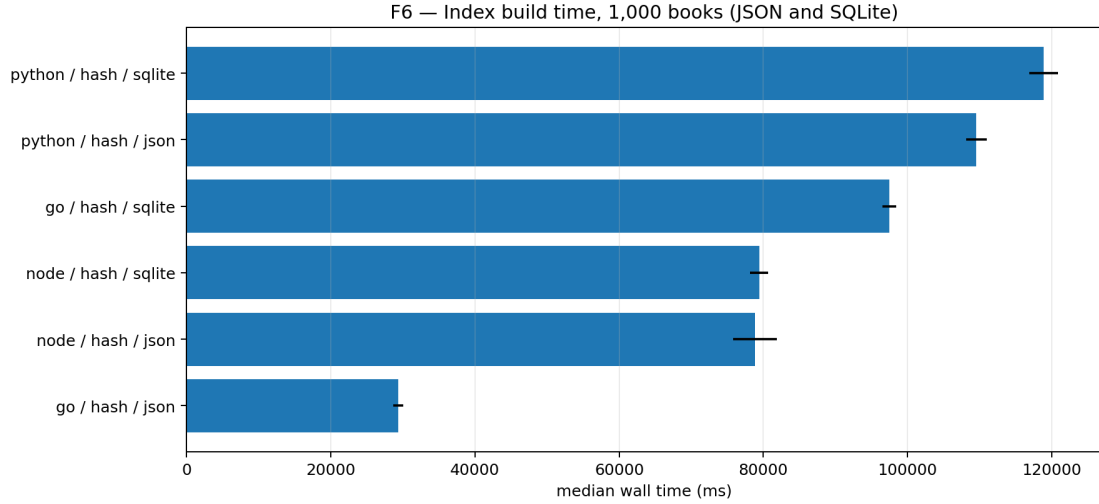


Figure 7: F6: index-build time for JSON and SQLite over 1,000 books.

Query latency is shown in Figure 8. These micro-benchmark results are currently available for Python. JSON and SQLite are almost identical for a single-term query (27.3 vs. 28.2 μ s). SQLite is somewhat faster for the AND-2 and AND-3 workloads (2.20 vs. 2.42 ms and 3.61 vs. 3.87 ms), while JSON is faster for the absent-term workload (3.6 vs. 7.9 μ s). Consequently, neither backend dominates every query type.

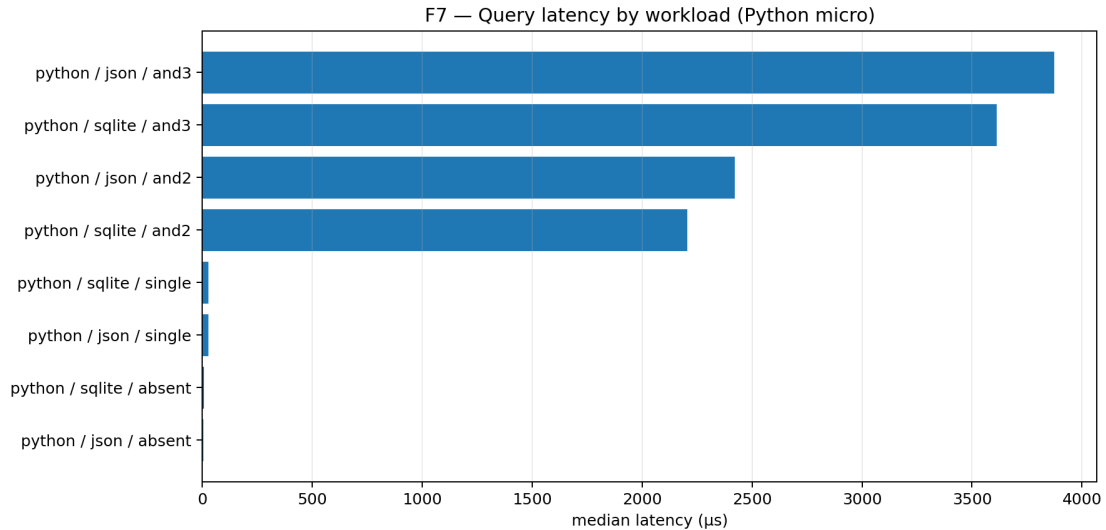


Figure 8: F7: query latency by workload (Python micro-benchmark).

For a 50-book update on top of the 1,000-book index, Figure 9 again shows different trade-offs. Go JSON is fastest at 8.6 s, followed by Node SQLite at 12.9 s. Go SQLite

takes 19.3 s, Node JSON 23.0 s, and both Python configurations are about 26 s. The Node result is noteworthy because SQLite is substantially faster than JSON for the update even though their full-build times are almost equal.

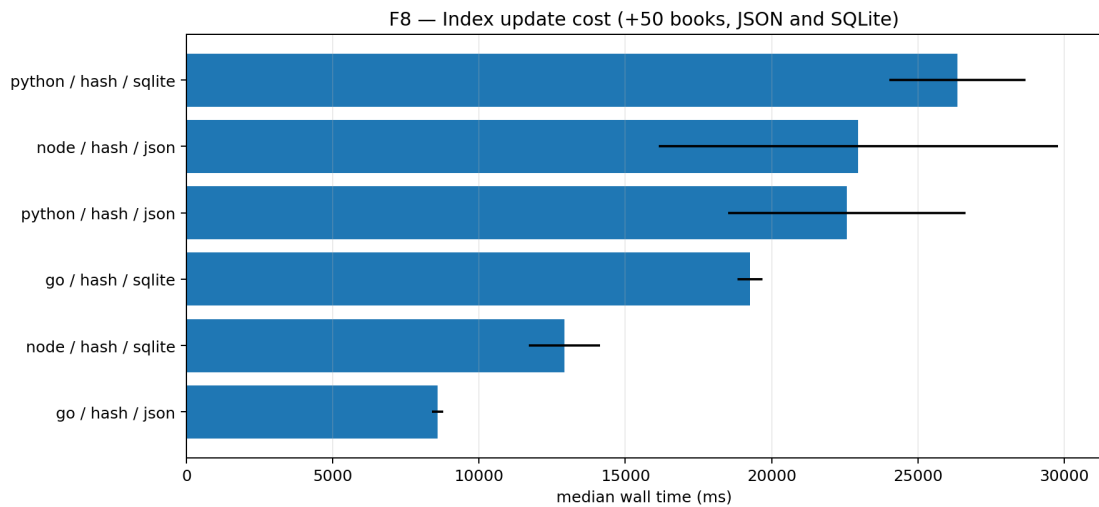


Figure 9: F8: update cost for 50 additional books.

Peak memory during full index construction is shown in Figure 10. The fastest configuration, Go JSON, is also the most memory-hungry at about 2254 MiB. Python SQLite uses the least memory at about 451 MiB. Node SQLite uses about 762 MiB, substantially less than Node JSON at about 1600 MiB.

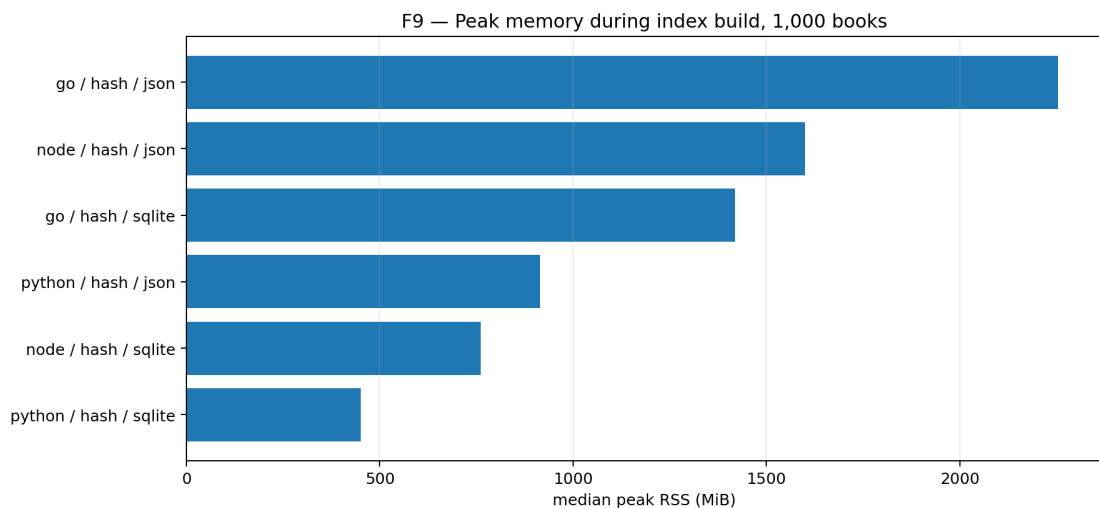


Figure 10: F9: peak RSS during index construction.

Figure 11 combines build time and memory in a Pareto analysis. The measured Pareto front contains Go JSON, Node JSON, Node SQLite, and Python SQLite. Go SQLite and Python JSON are dominated because another measured configuration is both faster and more memory efficient. This confirms that a single “fastest” result is not enough to choose a backend.

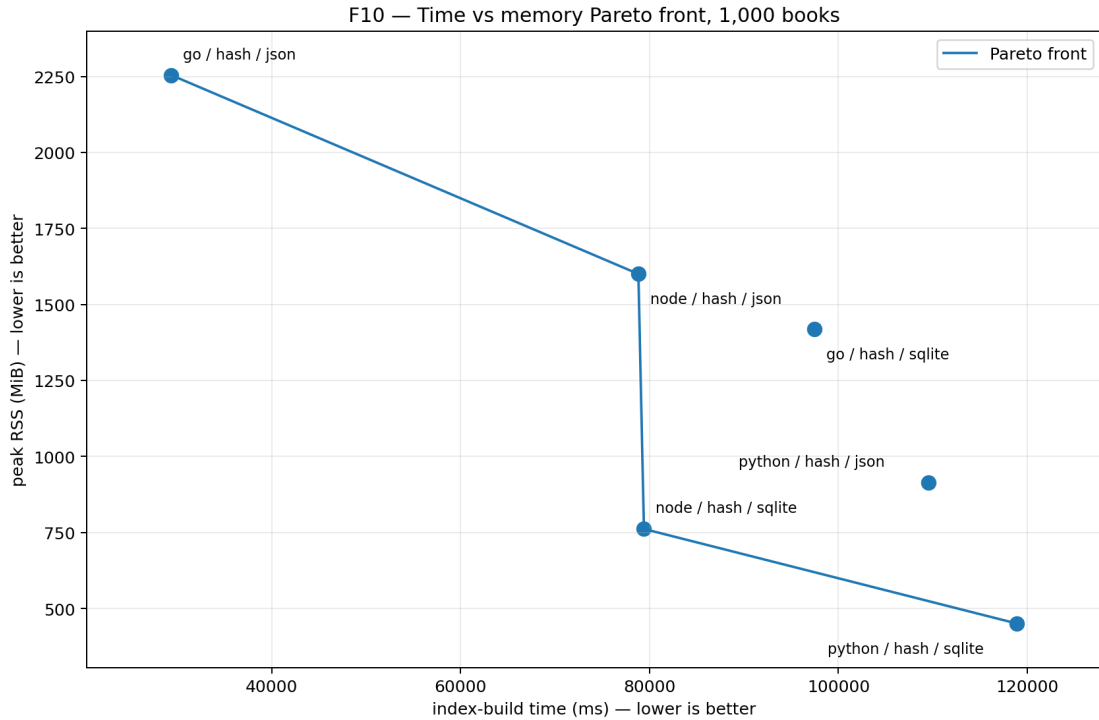


Figure 11: F10: time–memory Pareto front for 1,000-book index construction.

Folder backend and scalability limit. The folder backend exposes the cost of the one-file-per-term design. Even at only 100 books, E6 takes 523.5 s in Go, 584.8 s in Python, and 3084.9 s in Node. Updating that index with 50 additional books takes 618.6 s, 672.3 s, and 2108.5 s, respectively. This confirms the prediction that repeated small-file creation and fsync calls would dominate the workload. Running the same backend at 1,000 or 10,000 books would have required hours and potentially around a million files, so it was not pursued.

Selected index backend. For the default Stage 1 configuration we select **SQLite**. JSON achieves the best raw build and update times in Go, but it does so with high memory consumption and requires rewriting a monolithic representation. SQLite provides more stable update semantics and a much better memory profile in Python and Node. In particular, Node SQLite is almost as fast as Node JSON for a full build, uses less than half the memory, and is substantially faster for the 50-book update. Folder storage is rejected as the default because its file-per-term durability strategy is already prohibitively slow at 100 books.

5.4 Supplementary Metadata Datamart Results

The SQLite metadata datamart was also tested as a supplementary experiment. At the 1,000-book tier, metadata construction remains on the order of a few seconds across the three languages and layouts. Batch size 500 is generally modestly faster than batch size 1; however, the difference is limited because much of the cost comes from reading bodies and computing SHA-256 hashes rather than from inserting rows.

Metadata query results also show the expected effect of indexes. Between 100 and

1,000 books, indexed lookups remain in the low-microsecond range. In contrast, the case-insensitive `LIKE prefix%` title query rises from 6.2 to 49.4 μ s because SQLite scans the table. A range formulation uses the title index and remains around 4 μ s, but it is case-sensitive and therefore is not semantically interchangeable with the original query.

5.5 Results Compared with Expectations

Several results confirm the design expectations. The folder backend was expected to be expensive because it performs one durable write per term; the benchmark confirms this very strongly. SQLite was expected to trade some raw speed for structured updates and lower memory use, which is also visible in the results.

Two observations are more informative than the initial expectations. First, increasing the worker count does not improve Node ingestion, unlike Python and Go; synchronous filesystem work serializes enough of the Node path to eliminate the expected concurrency benefit. Second, SQLite is not uniformly slower than JSON: Node SQLite is substantially faster for the incremental index update. These contradictions are therefore explained by the implementation and workload rather than treated as benchmark errors.

5.6 Threats to Validity

The results were collected on a single laptop, one NVMe SSD, one ext4 filesystem under WSL2, and one set of runtime versions. Absolute timings therefore should not be generalized to different hardware or filesystems. In particular, the folder backend is especially sensitive to filesystem behavior.

The corpus is English-only and the largest fully evaluated mandatory tier is 1,000 books. The planned 10,000-book E10/E11 scaling run was not completed, and SQLite scalability beyond the measured tier is therefore not established. The folder backend is measured only at 100 books because its one-file-per-term design was already extremely slow at that size.

The local mirror deliberately removes WAN latency and Project Gutenberg rate limiting. This improves reproducibility but means the ingestion benchmark does not represent real-world Internet download time. The number of repetitions is also modest, so medians and IQR reduce sensitivity to outliers but do not eliminate run-to-run uncertainty.

Finally, E2 lookup and E7 query micro-benchmarks are available only for Python, so those figures should be interpreted primarily as comparisons between layouts/backends rather than as cross-language results. SQLite is used in place of MongoDB, so conclusions about the database-backed design apply to an embedded relational store rather than to distributed or client-server databases.

6 Conclusions and Future Improvements

Stage 1 establishes a reproducible data layer shared by Python, Node.js, and Go. The cross-language implementation uses the same data source, tokenizer rules, positional index semantics, control state, and canonical output contract, allowing the benchmark to compare storage and runtime choices rather than different functionality.

The measurements lead to two concrete default choices. For the datalake, the **batch/prefix layout** provides the best overall balance: direct lookup is cheap, parallel ingestion remains competitive, directory fan-out is bounded, and storage cost is almost identical to the other layouts. For the inverted index, **SQLite** is selected as the default because it provides a substantially better memory profile and robust update behavior while retaining competitive performance. JSON remains attractive when raw speed is the only objective, especially in Go, whereas the folder backend is not practical at larger corpus sizes under the current durability strategy.

The results also demonstrate why the three-language comparison is useful. Go with JSON delivers the fastest index construction and update, Node SQLite provides a strong time-memory compromise, and Python SQLite minimizes memory consumption. No single language/backend combination dominates every resource dimension.

Stage 2 can build ranking methods such as TF-IDF or BM25, phrase-aware queries, and richer search functionality on top of the positional index already produced in Stage 1. Future work should complete the 10,000-book scalability tier, add Node and Go micro-benchmarks for lookup and query, investigate asynchronous/batched Node ingestion, evaluate a less fsync-heavy folder-index update strategy, and test the selected designs on additional hardware and filesystems.